System Design Documentation for LibraQ

Version 2.0

Prepared by: Taisia, Timothy, Akhanali, Parnia

November 16th 2025

Table of Contents:

CRC cards..... 3
Description of system interaction with the environment..... 10
Operating Environment:..... 10
Data Storage:..... 10
External Dependencies..... 11
Assumptions and constraints:..... 11
Architecture of the system..... 11
Dealing with errors..... 11
Input and User Errors:..... 11
Business Logic Errors..... 12

CRC cards

Enum:

Class Name: Genre	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • List different Genres and their associated display name.	**Collaborators:** • None

Class Name: RequestStatus	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • List different status of values of the book request.	**Collaborators:** • None

Class Name: ReservationStatus	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • List different status of values of the reservation.	**Collaborators:** • None

Entities:

Class Name: User (abstract)	
Parent Class (if any): None **Subclasses (if any):** Librarian, Renter	
Responsibilities: • Store user information (username, password, name, email).	**Collaborators:** • None

Class Name: Librarian
Parent Class (if any): User

Subclasses (if any): None	
Responsibilities: ● Store attributes and permissions specific to librarians.	**Collaborators:** ● None

Class Name: Renter	
Parent Class (if any): User **Subclasses (if any):** None	
Responsibilities: ● Store attributes and permissions specific to Renters	**Collaborators:** ● None

Class Name: Book	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Store all book attributes (title, author, ISBN, genre, availability).	**Collaborators:** ● Genre

Class Name: CheckoutReceipt	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Store all attributes corresponding to a checkout receipt, including the user and book, the date the book was checked out, the due date, date book was returned, and fine amount associated with that checkout.	**Collaborators:** ● User ● Book

Class Name: Reservation	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Store all attributes corresponding	**Collaborators:** ● User

to a reservation, including the state of the reservation,	• Book • ReservationStatus

Class Name: BookRequest	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Store all data regarding a Book Request submitted by a renter, including the title, author, user, status and optional notes, and also date requested.	**Collaborators:** • RequestStatus • User

Repositories:

Class Name: BookRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of Book objects. • Provide methods for finding a book based on specific attributes. • Provide methods for adding/removing books.	**Collaborators:** • Book

Class Name: UserRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of User objects. • Provide methods for finding a user based on specific attributes. • Provide a method for adding Users.	**Collaborators:** • User

Class Name: CheckoutReceiptRepository

Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of CheckoutReceipt objects. • Provide methods for finding a CheckoutReceipt based on User or Book. • Provide a method for adding a new receipt to DB.	**Collaborators:** • CheckoutReceipt • ReservationRepository • User • Book

Class Name: BookRequestRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of BookRequest objects. • Provide methods for finding a BookRequest based on Requestor(Renter), Status, date requested, and by request id.	**Collaborators:** • BookRequest • User • RequestStatus

Class Name: ReservationRepository	
Parent Class (if any): JpaRepository (interface) **Subclasses (if any):** None	
Responsibilities: • Store and manage a collection of Reservation objects. • Provide methods for finding a Reservation based on User, Book or Status (or combination).	**Collaborators:** • Reservation

Controllers:

Class Name: BookController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Loading the books detail page for a specific book	**Collaborators:** • BookService • CheckoutService

• Calling CheckoutService when a user clicks on the checkout button on the page • Display correct availability for a book • Provide option for a user to Reserve an unavailable book • Allow librarian to check in book	• UserService • ReservationService

Class Name: HomeController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Responsible for loading the home page using appropriate services (loading the book list using BookService). • Routing clicks on books to the appropriate book details page.	**Collaborators:** • BookService

Class Name: RegistrationController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Handle the User registration form submission • Validates User;s input data(email format, password strength and match) • Redirects to Home page on successful registration	**Collaborators:** • UserService • Renter

Class Name: AuthenticationController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Display the log in form page	**Collaborators:** • SecurityConfig

Class Name: UserDetailsController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Gets currently logged in User's data • Gets checkout receipts for currently logged in user	**Collaborators:** • User • CheckoutReceipt

Class Name: BookRequestController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Display Book Request form. • Handle book request form submission(validating data in the form, creating BookRequest) • Display User's(Renter's) book requests • Display Book Requests management page for the librarian(calculating the stats, filtering requests) • Handle approve book requests • Handle reject book requests	**Collaborators:** • BookRequest • Request Status • User • BookRequestService • UserService

Class Name: LibrarianConsoleController	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Handle adding Books • Handle removing Books • Display Pending Book Requests	**Collaborators:** • BookRequest • Request Status • User • BookRequestService • BookService • UserService

Services:

Class Name: BookService	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Handle search queries of books. ● Provide methods for adding/removing books.	**Collaborators:** ● Book ● BookRepository

Class Name: CheckoutService	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Manage book checkouts and checkout extensions for renters. ● Handle check-ins when librarians process returns. ● Update availability status of Books. ● Manage reservations of books by renters.	**Collaborators:** ● BookRepository ● CheckoutReceiptRepository ● User, Renter ● Book

Class Name: UserService	
Parent Class (if any): UserDetailsService **Subclasses (if any):** None	
Responsibilities: ● Provides User CRUD operations ● Implements UserDetailsService for authentication ● Loads User by email for login ● Determines User's role for authorization	**Collaborators:** ● User ● UserRepository ● PasswordEncoder

Class Name: BookRequestService	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: ● Saves the BookRequests into the	**Collaborators:** ● BookRequest

repository • Retrieve BookRequests(all, by status, by requestor) • Approve and Reject the book requests • Delete a BookRequest	• RequestStatus • User • BookRequestRepository

Class Name: ReservationService	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Manage book reservations. • Change status of reservation when it is created, when book is returned and when reservation is fulfilled or expired. • Provide methods for querying reservations based on Book, User, or ReservationStatus, or combination.	**Collaborators:** • ReservationRepository • User • Book • ReservationStatus

Configuration:

Class Name: SecurityConfig	
Parent Class (if any): None **Subclasses (if any):** None	
Responsibilities: • Configures the authentication and authorization systems • Configures the log in processes • Configures successful log in redirect • Configures the log out logic • Configures public and protected URL's	**Collaborators:** • SpringSecurity • UserService • AuthenticationController

Description of system interaction with the environment

Operating Environment:

- **Operating System:** Cross-platform (Windows, macOS, or Linux)
- **Development Environment:** Eclipse IDE for Enterprise Java Developers
- **Programming Language:** Java 17
- **Framework:** Spring Boot (version 3.x)
- **Build Tool:** Maven
- **Browser Support:** Any modern web browser (e.g., Chrome, Firefox, Edge) can be used to access the web interface.

Data Storage:

- Data is stored using Springboot H2 Database.
- In the file system it is actually stored in data/librarydb.mv.db file

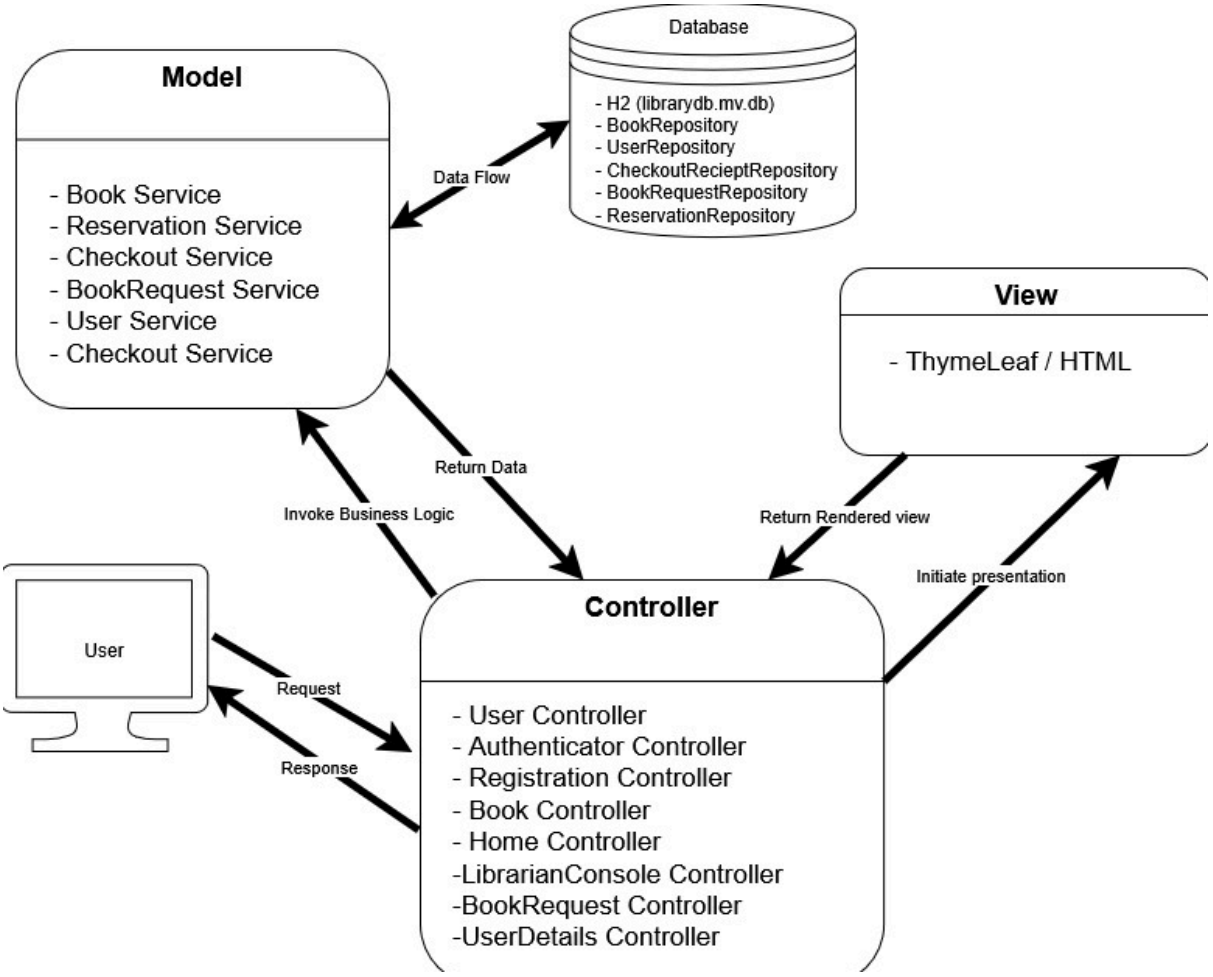
External Dependencies

- Thymeleaf – for rendering server-side HTML templates that display dynamic content.
- H2 Database
- Spring Security

Assumptions and constraints:

- The system assumes a single active local deployment (no distributed or concurrent multi-user setup).
- Network connectivity is not required, since the application runs fully offline on localhost.

Architecture of the system



Dealing with errors

Input and User Errors:

Examples: Invalid login credentials.
Handling:

- Inputs are validated and Controllers catch exceptions such as InvalidInputException or BookNotFoundException and return messages that will be displayed to the user (e.g., “Book not available”, “Invalid credentials”).
- Controllers validate inputs using Jakarta Bean Validation (@Valid, @NotBlank, @Email, @Size)

Business Logic Errors

Examples: Trying to check out a book that’s already checked out, or reserving a book twice.
Handling:

- CheckoutService and ReservationService validate all operations before performing updates.

- Violations throw custom exceptions, caught by controllers and shown as alerts.
-
-